
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Week 4a: Hardware-Aware Reverse Engineering with CMSIS-SVD: Live GDB and Ghidra Analysis of Stripped Binaries

ARM Cortex-M33 • GDB • Ghidra

September 25, 2026

For educational and defensive security research only

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

What You'll Learn This Week

By the end of this tutorial, you will be able to:

- Understand the fundamental differences between an ELF binary and a raw `.bin` firmware dump.
- Flash and debug `CTF-01.bin` using OpenOCD and a hardware Debug Probe with explicit base addressing (`0x10000000`).
- Operate GDB without an executable or symbol table, using hardware breakpoints (`hb`) and address-based disassembly (`x/i`).
- Navigate the ARM Cortex-M Vector Table at `0x10000000` to find the initial Stack Pointer (`0x20082000`) and Reset Vector (`0x1000015B` → `0x1000015A`).
- Follow boot execution into `main()` at `0x100001E0` without debug symbols.
- Identify the Memory-Mapped I/O (MMIO) peripheral blind spot in stripped binaries (UARTo, IO_BANKo, PADS_BANKo, RESETS, SIO).
- Install and configure CMSIS-SVD (`rp2350.svd`) and `PyCortexMDebug` inside GDB for live dynamic peripheral inspection.
- Inspect UARTo baud rate and FIFO status registers, and verify GPIO 0/1 pin multiplexing live over SWD.
- Install and run `SVD-Loader-Ghidra` on `CTF-01.bin` to map peripheral memory blocks and auto-generate C struct definitions.
- Transform raw, obscure pointer decompilation into readable, vendor-grade peripheral struct accesses.
- Understand why arithmetic immediate instructions like `add.w r0, r0, #0x40000000` appear in the assembly listing while resolving cleanly in the decompiler.
- Execute a unified hardware-aware reverse engineering workflow combining static analysis in Ghidra and dynamic analysis in GDB to solve the CTF mission.

Part 1: The Raw Binary Dilemma (.elf vs .bin)

ELF Files: The Friendly Development Container

In previous lessons, when debugging your firmware, you launched GDB pointing to an **Executable and Linkable Format (ELF)** file:

```
arm-none-eabi-gdb build\CTF-01.elf
```

An ELF file is not just machine code. It is a rich, structured container that contains:

1. **ELF Header:** Specifies the target architecture, endianness, and exact entry point address.
2. **Section Headers:** Defines memory segments such as `.text`, `.data`, `.rodata`, and `.bss`.
3. **Symbol Table:** Maps human-readable names (`main`, `grid_deviation`, `evaluate_grid`) to exact virtual memory addresses.
4. **DWARF Debug Information:** Links individual assembly instructions back to original C source file line numbers, variable types, and stack frame layouts.

When GDB loads an ELF file, it knows the name of every function, the layout of every struct, and the location of `main`.

Raw .bin Files: The Harsh Reality of Firmware Extraction

In real-world hardware reverse engineering, red teaming, and firmware extraction (such as reading an external SPI flash chip or intercepting an over-the-air firmware update), you will almost never have access to an ELF file.

Instead, you are handed a **raw binary file (.bin)**:

Comparison: ELF Container vs. Raw .bin Firmware	
CTF-01.elf (Development Build)	
ELF Header (Entry Point: 0x1000015b)	
Symbol Table (main -> 0x100001e0, evaluate_grid -> ...)	
DWARF Debug Data (C Source Line Mapping)	
Section Table (.text, .rodata, .data, .bss)	
Machine Code Payload (15,920 bytes)	
CTF-01.bin (Field Recovery Image)	
[Flat Machine Code Bytes Only - No Headers, No Symbols]	
00 20 08 20 5b 01 00 10 1b 01 00 10 1d 01 00 10 ...	

A `.bin` file is a byte-for-byte memory dump of flash memory. It contains:

- **No ELF header**
- **No section names**
- **No symbol names**
- **No debug information**
- **No base address metadata** (the file itself does not record where in memory it belongs)

If you attempt to launch GDB with `0x0001b_ctf/CTF-01.bin` directly:

```
arm-none-eabi-gdb 0x0001b_ctf\CTF-01.bin
```

GDB immediately halts with an error:

```
"0x0001b_ctf/CTF-01.bin": not in executable format: file format not recognized
```

The Hardware MMIO Blind Spot

Stripped binaries introduce a second, even larger obstacle: **hardware peripherals**.

Microcontrollers interact with the outside world using **Memory-Mapped Input/Output (MMIO)**.

Peripherals like UART, GPIO, Clocks, and Resets do not have special CPU instructions. Instead, they are mapped to specific, fixed physical addresses in the microcontroller's memory map:

Subsystem	RP2350 Physical Address Base	Function
RESETS	0x40020000	Subsystem reset controller (releases UART and GPIO from reset)
IO_BANK0	0x40028000	GPIO pin function select (FUNCSEL) and overrides
PADS_BANK0	0x40038000	Electrical pad controls (drive strength, pulls, Schmitt)
UART0	0x40070000	Serial UART interface (115200 8N1 telemetry console)
SIO	0xd0000000	Single-cycle I/O (fast CPU core check, GPIO controls)

When analyzing a raw binary, neither GDB nor Ghidra understands what `0x40070018` or `0x40028004` mean. To standard tools, they are just arbitrary hexadecimal numbers. Reverse engineers are left manually cross-referencing thousands of pages of microcontroller datasheets.

In this tutorial, you will master the two industry-standard tools that eliminate this blind spot:

1. **PyCortexMDebug** for live, dynamic hardware awareness in **GDB**.
2. **SVD-Loader-Ghidra** for automatic struct mapping and clear decompilation in **Ghidra**.

Part 2: The Target Application: `0x0001b_ctf` (Operation Black Start)

Mission Briefing

Our primary target is the emergency firmware build from `0x0001b_ctf` (**CTF-01: Operation Black Start**).

In this scenario, a critical cyberattack severed the primary SCADA network coordinating regional power grid interconnections. An emergency firmware build was deployed to the **GRID-7** fleet of relay controllers to manage an automated black-start restoration. However, the rushed build contains two critical defects:

1. **Miscalibrated Safety Threshold:** The frozen grid frequency deviation latched at 0.87 Hz is evaluated against a corrupt threshold of 95 (0.95 Hz) instead of the hard engineering safety limit of 60 (0.60 Hz), causing the relay to report a false-safe `STABLE` status.
2. **Hardcoded False Status Line:** The operator console line falsely asserts `SIGNAL: NORMAL` regardless of actual channel health.

The source code was overwritten during the crisis build process. The only surviving artifact is the compiled raw binary: `CTF-01.bin`.

The Source Code Behind the Binary

Let's review the firmware architecture from `0x0001b_ctf/src/` to understand what machine code the compiler generated:

`main.c`

```
#include "console.h"
#include "grid.h"
#include "pico/stdlib.h"

int main(void)
{
    stdio_init_all();
    retain_dispatch_frame();
    evaluate_grid();
    print_boot_banner();
    while (true) {
        print_status();
        sleep_ms(1000);
    }
    return 0;
}
```

`grid.c`

```
#include "grid.h"
#include <stdint.h>

// Frozen grid frequency deviation reading latched when comms were severed.
volatile uint32_t grid_deviation = 87;

// Operator-facing classification of the frozen reading (drives GRID STATUS).
volatile uint32_t operator_state = 0;

// Automated dispatch authorization decision (drives DISPATCH PATH).
volatile uint32_t dispatch_state = 0;

// Quarantined black-start authorization frame, retained in flash, never sent.
static volatile const char dispatch_frame[] =
    "WORLDGRID:BLACKSTART:GRID-7:WATER-3";

void retain_dispatch_frame(void)
{
    volatile char frame_marker = dispatch_frame[0];
    (void)frame_marker;
}

void evaluate_grid(void)
{
    operator_state = (grid_deviation < SAFE_THRESHOLD) ? 1 : 0;
    dispatch_state = (grid_deviation < SAFE_THRESHOLD) ? 1 : 0;
}
```

console.c

```

#include "console.h"
#include "grid.h"
#include <stdio.h>

void print_boot_banner(void)
{
    printf("GLOBAL EMBEDDED RESPONSE NETWORK\r\n");
    printf("BLACK START WINDOW: 27 MINUTES\r\n");
    printf("UART0 115200 8N1 | AUTHORIZED LAB CONSOLE\r\n");
    printf("SIGNAL: NORMAL\r\n");
    printf("RESPONSE> ");
}

void print_status(void)
{
    printf("GRID STATUS: %s\r\n", operator_state ? "STABLE" : "CRITICAL");
    printf("DISPATCH PATH: %s\r\n", dispatch_state ? "AUTHORIZED" : "HELD");
    printf("LAST FRAME: QUARANTINED\r\n");
    printf("RESPONSE> ");
}

```

Hardware Actions Performed by This Code

1. `stdio_init_all()`:
 - Releases the UART0, IO_BANK0, and PADS_BANK0 peripherals from reset via `RESETS` (`0x40020000`).
 - Configures `GPI00` (TX) and `GPI01` (RX) pin multiplexing to function select 2 (UART0) in `IO_BANK0` (`0x40028000`).
 - Configures pad electrical properties in `PADS_BANK0` (`0x40038000`).
 - Programs baud rate divisors (`UARTIBRD`, `UARTFBRD`) and line controls (`UARTLCR_H`) in `UART0` (`0x40070000`) for 115200 baud, 8 data bits, no parity, 1 stop bit (8N1).
2. `retain_dispatch_frame()`:
 - Anchors the secret authorization token string `"WORLDGRID:BLACKSTART:GRID-7:WATER-3"` at physical flash address `0x100037A0`.
3. `evaluate_grid()`:
 - Compares the global `grid_deviation` variable (87) against `SAFE_THRESHOLD` (compiled as `cmp r3, #94`).
4. `print_boot_banner()` & `print_status()`:
 - Streams formatted strings through the UART0 Transmit FIFO buffer (`UART0_UARTDR`).

In your workspace, this challenge is located in `0x0001b_ctf/`:

- `0x0001b_ctf/CTF-01.bin` (Raw 15,920-byte stripped firmware)
- `0x0001b_ctf/CTF-01.uf2` (Packaged UF2 image)

For the remainder of this lesson, we assume you **only have** `0x0001b_ctf/CTF-01.bin`.

Part 3: Flashing and Connecting OpenOCD with Raw Binaries

Flashing a Raw Binary via OpenOCD

When flashing an `.elf` file, OpenOCD reads target memory addresses directly from the ELF program headers. However, because a `.bin` file contains no header information, you **must explicitly specify the physical base address** (0x10000000 for RP2350 XIP flash).

Open a PowerShell terminal in your repository root:

```
& "$env:USERPROFILE\.pico-sdk\openocd\0.12.0+dev\openocd.exe" `
-s "$env:USERPROFILE\.pico-sdk\openocd\0.12.0+dev\scripts" `
-f interface/cmsis-dap.cfg `
-f target/rp2350.cfg `
-c "adapter speed 5000; program 0x0001b_ctf/CTF-01.bin 0x10000000 verify reset exit"
```

Notice the crucial parameter: `0x10000000`. This instructs OpenOCD to write the binary bytes starting at the exact beginning of external flash memory.

```
** Programming Started **
[rp2350.dap.core0] target halted due to debug-request
wrote 16384 bytes from file 0x0001b_ctf/CTF-01.bin in 0.412s
** Programming Finished **
** Verify Started **
verified 15920 bytes in 0.082s
** Verified OK **
** Resetting Target **
shutdown command invoked
```

Starting OpenOCD as a Live Debug Server

To debug the running firmware interactively, launch OpenOCD without the `exit` command. Keep this terminal open:

```
& "$env:USERPROFILE\.pico-sdk\openocd\0.12.0+dev\openocd.exe" `
-s "$env:USERPROFILE\.pico-sdk\openocd\0.12.0+dev\scripts" `
-f interface/cmsis-dap.cfg `
-f target/rp2350.cfg `
-c "adapter speed 5000; init"
```

OpenOCD initializes the SWD hardware connection and listens for GDB connections on TCP port `3333`.

```
Info : Hardware thread awareness created
Info : Listening on port 3333 for gdb connections
```

Part 4: Dynamic Analysis in GDB Without an ELF

Because we are analyzing a raw binary, we do not pass a file name to GDB on startup.

Open a second terminal window:

```
arm-none-eabi-gdb
```

Step 1: Set Target Architecture and Connect

Inside the GDB prompt, tell GDB what processor architecture to expect, connect over TCP to OpenOCD, and immediately halt the processor:

```
set architecture armv8-m.main
target extended-remote :3333
monitor reset halt
```

```
The target architecture is assumed to be armv8-m.main
Remote debugging using :3333
target halted due to debug-request, current mode: Thread
xPSR: 0x01000000 pc: 0x1000015a msp: 0x20082000
```

Step 2: Decode the Hardware Vector Table

How did the CPU know to halt at `0x1000015a`?

On ARM Cortex-M processors, the first entries of flash memory (`0x10000000`) define the **Vector Table**:

- **Word 0** (`0x10000000`): Initial Main Stack Pointer (`msp`).
- **Word 1** (`0x10000004`): Reset Vector (address of the first instruction to run upon reboot).

Let's inspect the first four 32-bit words of flash using GDB's examine command (`x/4wx`):

```
x/4wx 0x10000000
```

```
0x10000000: 0x20082000 0x1000015b 0x1000011b 0x1000011d
```

Let's analyze what these words reveal:

```
+-----+
| RP2350 Cortex-M33 Vector Table Decoding |
+-----+
| Address      Value      Meaning          |
| 0x10000000:  0x20082000  Initial MSP (Top of 512KB SRAM) |
| 0x10000004:  0x1000015b  Reset Vector (Thumb Execution Bit 0) |
|                                     Execution begins at: 0x1000015a |
+-----+
```

[!NOTE] The Reset Vector value is `0x1000015b`. In ARM architecture, bit 0 indicates Thumb instruction mode (`0x1000015A + 1`). The processor automatically clears bit 0 and begins executing Thumb instructions at address `0x1000015a`.

Step 3: Why Symbolic Commands Fail

If you try to use symbolic commands, GDB cannot help you:

```
break main
disassemble main
```

No symbol table is loaded. Use the "file" command.

There are no symbols! To reverse engineer this firmware dynamically, we must navigate using **instruction addresses**.

Step 4: Disassemble by Address Range

Disassemble 12 instructions starting at the Reset Handler address (0x1000015A):

```
x/12i 0x1000015a
```

```
=> 0x1000015a: mov.w    r0, #3489660928 ; 0xd0000000 (SIO base)
0x1000015e: ldr     r0, [r0, #0]    ; SIO_CPUID (reads core id)
0x10000160: cbz     r0, 0x10000166  ; if Core 0, branch to init
0x10000162: movs    r0, #0
0x10000164: b.n     0x1000014e      ; Core 1 sleeps
0x10000166: add     r4, pc, #52     ; loads init table pointer
0x10000168: ldmia   r4!, {r1, r2, r3}
0x1000016a: cmp     r1, #0
0x1000016c: beq.n   0x10000174
0x1000016e: bl      0x10000196      ; copy data section to SRAM
0x10000172: b.n     0x10000168
0x10000174: ldr     r1, [pc, #84]   ; loads BSS bounds
```

Notice instruction `0x1000015a`: it immediately reads SIO register `0xd0000000` (CPUID) to check whether execution is occurring on Core 0 or Core 1!

Now look further down the boot sequence at address `0x10000186`:

```
x/6i 0x10000184
```

```
0x10000184: blx     r1
0x10000186: ldr     r1, [pc, #80]   ; loads address from 0x100001d8
0x10000188: blx     r1              ; calls main()!
0x1000018a: ldr     r1, [pc, #80]
0x1000018c: blx     r1
0x1000018e: bkpt    0x0000
```

Inspect the pointer stored at `0x100001d8`:

```
x/wx 0x100001d8
```

```
0x100001d8: 0x100001e1
```

Value `0x100001e1` is Thumb address `0x100001E0 + 1`. This reveals that `main()` is located at address `0x100001e0`!

Step 5: Setting Hardware Breakpoints on Flash Memory

In RAM, GDB can set software breakpoints by temporarily replacing instructions with a breakpoint opcode (`bkpt`). However, external XIP flash (`0x10000000` – `0x1FFFFFFF`) is **read-only**. GDB cannot write to flash memory while the processor is running.

Therefore, you must use **hardware breakpoints** (`hb`):

```
hb *0x100001e0
continue
```

```
Hardware assisted breakpoint 1 at 0x100001e0
Continuing.
```

```
Breakpoint 1, 0x100001e0 in ?? ()
```

We are now stopped at the entry point of `main()` inside a completely stripped binary!

Disassemble the instructions in `main()` from `0x100001e0` to `0x10000216`:

```
disassemble 0x100001e0, 0x10000216
```

```
Dump of assembler code from 0x100001e0 to 0x10000216:
=> 0x100001e0: push    {r7, lr}
    0x100001e2: sub     sp, #8
    0x100001e4: bl      0x1000308c      ; stdio_init_all()
    0x100001e8: ldr     r3, [pc, #124]   ; [0x10000268] -> 0x100037a0 (dispatch_frame)
    0x100001ea: ldr     r2, [pc, #128]   ; [0x1000026c] -> 0x200005d8 (grid_deviation)
    0x100001ec: ldrb    r3, [r3, #0]     ; retain_dispatch_frame(): reads dispatch_frame[0] ('W')
    0x100001ee: ldr     r6, [pc, #128]   ; [0x10000270] -> 0x20000844 (operator_state)
    0x100001f0: strb.w  r3, [sp, #7]     ; store frame_marker
    0x100001f4: ldrb.w  r3, [sp, #7]     ; reload frame_marker (volatile)
    0x100001f8: ldr     r3, [r2, #0]     ; evaluate_grid(): read grid_deviation (87)
    0x100001fa: ldr     r5, [pc, #120]   ; [0x10000274] -> 0x20000834 (dispatch_state)
    0x100001fc: cmp     r3, #94          ; compare site A: if grid_deviation <= 94
    0x100001fe: ite     hi
    0x10000200: movhi   r3, #0
    0x10000202: movlsl  r3, #1
    0x10000204: str     r3, [r6, #0]     ; operator_state = 1 (STABLE)
    0x10000206: ldr     r3, [r2, #0]     ; read grid_deviation again
    0x10000208: ldr     r0, [pc, #108]   ; string pointer
    0x1000020a: cmp     r3, #94          ; compare site B: if grid_deviation <= 94
    0x1000020c: ite     hi
    0x1000020e: movhi   r3, #0
    0x10000210: movlsl  r3, #1
    0x10000212: str     r3, [r5, #0]     ; dispatch_state = 1 (AUTHORIZED)
    0x10000214: bl      0x1000311c      ; puts("GLOBAL EMBEDDED RESPONSE NETWORK")
End of assembler dump.
```

Notice what reverse engineering revealed right before our eyes:

- `0x100001e4` : `bl 0x1000308c` initializes UART0 serial communications.
- `0x100001e8` : Loads pointer `0x100037a0` . Inspecting `x/s 0x100037a0` reveals the secret token `"WORLDGRID:BLACKSTART:GRID-7:WATER-3"` !
- `0x100001fc` & `0x1000020a` : Both threshold checks compare `r3` against `#94` (`0x5e`). Because `grid_deviation` is `87`, $87 \leq 94$, so both `operator_state` and `dispatch_state` are incorrectly set to 1!

Step 6: The MMIO Blindness in Action

Now step past `0x100001e4` (`stdio_init_all`) using `nexti` or `stepi`.

In standard GDB, if you want to inspect what happened to UART0, you are forced to type raw hex addresses:

```
x/wx 0x40070018
```

```
0x40070018: 0x00000090
```

What does `0x90` mean?

- Is the Transmit FIFO empty?
- Is the Receive FIFO full?
- Is the UART transmitter currently busy?
- What baud rate divisor was written to `0x40070024` ?

Standard GDB has no way to tell you. Let's fix that right now.

Part 5: Installing and Configuring CMSIS-SVD in GDB (PyCortexMDebug)

What is CMSIS-SVD?

CMSIS-SVD (Common Microcontroller Software Interface Standard - System View Description) is an open XML specification developed by ARM. Silicon vendors publish an `.svd` file for every chip they manufacture.

An SVD file contains a complete, machine-readable description of:

- Every peripheral on the chip
- Its physical base address
- Every register and offset
- Every bitfield, bitmask, access permission, and human-readable description

```
+-----+
| CMSIS-SVD Hierarchy Tree |
+-----+
| Device: RP2350           |
| +- Peripheral: UART0 (Base: 0x40070000) |
| |   +- Register: UARTDR (Offset: 0x000) |
| |   +- Register: UARTFR (Offset: 0x018) |
| |   +- Register: UARTIBRD (Offset: 0x024) |
| |   +- Register: UARTLCR_H (Offset: 0x02c) |
| +- Peripheral: IO_BANK0 (Base: 0x40028000) |
| |   +- Register: GPIO0_CTRL (Offset: 0x004) -> UART0 TX |
| |   +- Register: GPIO1_CTRL (Offset: 0x00c) -> UART0 RX |
| +- Peripheral: PADS_BANK0 (Base: 0x40038000) |
| |   +- Register: GPIO0 (Offset: 0x004) |
| +- Peripheral: SIO (Base: 0xd0000000) |
| |   +- Register: CPUID (Offset: 0x000) |
+-----+
```

Step 1: Obtain `rp2350.svd`

A pre-downloaded copy of `rp2350.svd` is included directly in your course repository under `WEEK04/rp2350.svd`. You can also download the latest version directly from the official open-source `cmsis-svd-data` repository.

Open a PowerShell terminal and create a dedicated `svd` directory in your user profile:

```
New-Item -ItemType Directory -Force -Path "$env:USERPROFILE\svd"

# Option A: Copy from local course folder
Copy-Item "WEEK04\rp2350.svd" "$env:USERPROFILE\svd\rp2350.svd"

# Option B: Download directly from GitHub
Invoke-WebRequest `
  -Uri "https://raw.githubusercontent.com/cmsis-svd/cmsis-svd-`
  data/main/data/RaspberryPi/rp2350.svd" `
  -OutFile "$env:USERPROFILE\svd\rp2350.svd"
```

Verify that the file is in place:

```
Get-Item "$env:USERPROFILE\svd\rp2350.svd"
```

Step 2: Install `PyCortexMDebug`

`PyCortexMDebug` is an open-source Python extension for GDB created by Bill Nahill. It parses SVD XML files and exposes high-level peripheral inspection commands inside your GDB session.

Clone the repository into your user profile:

```
cd "$env:USERPROFILE"
git clone https://github.com/bnahill/PyCortexMDebug.git
```

Step 3: Load `PyCortexMDebug` in GDB

In your active GDB session, load the Python script and parse `rp2350.svd`:

```
source ~/PyCortexMDebug/scripts/gdb.py
svd_load ~/.svd/rp2350.svd
```

```
Loading SVD file /Users/username/.svd/rp2350.svd...
Loaded 52 peripherals
```

Just like that, GDB now understands every single register, offset, and bitfield on the RP2350 microcontroller!

[!TIP] You can automate this process so SVD support is always active. Add the following lines to your `~/.gdbinit` file:

```
source ~/PyCortexMDebug/scripts/gdb.py
svd_load ~/.svd/rp2350.svd
```

Part 6: Live Dynamic Peripheral Inspection & Manipulation in GDB

Now that GDB is hardware-aware, let's explore what we can do on our stripped `CTF-01.bin` firmware without touching source code.

Command 1: List All On-Chip Peripherals

Type `svd` with no arguments to list all 52 peripherals on the RP2350:

```
svd
```

Available Peripherals:

ACCESSCTRL	ADC	BUSCTRL	CLOCKS
DMA	GLITCH_DETECTOR	I2C0	I2C1
IO_BANK0	IO_QSPI	OTP	PADS_BANK0
PADS_QSPI	PIX_RP2040	PIO0	PIO1
PIO2	PLL_SYS	PLL_USB	POWMAN
PWM	QMI	RESETS	ROSC
SHA256	SIO	SPI0	SPI1
SYSINFO	SYSCFG	TBMAN	TICKS
TIMER0	TIMER1	TRNG	UART0
UART1	USB	VREG_AND_CHIP_RESET	WATCHDOG
XIP_AUX	XIP_CTRL	XIP_QMI	XOSC

Command 2: Inspect a Peripheral Live (`svd UART0`)

Inspect the **UART0** peripheral. GDB automatically reads the physical hardware registers across SWD and displays their live state:

```
svd UART0
```

UART0 @ 0x40070000:

```

UARTDR      : 0x00000000
UARTRSR     : 0x00000000
UARTFR      : 0x00000090
UARTILPR    : 0x00000000
UARTIBRD    : 0x00000043
UARTFBRD    : 0x00000035
UARTLCR_H   : 0x00000070
UARTCR      : 0x00000301
UARTIFLS    : 0x00000012
UARTIMSC    : 0x00000000
UARTRIS     : 0x00000000
UARTMIS     : 0x00000000
UARTICR     : 0x00000000
UARTDMACR   : 0x00000003

```

Look at what this dump proves:

- `UARTIBRD` (`0x43 = 67`) and `UARTFBRD` (`0x35 = 53`): These are the exact integer and fractional divisors for 115200 baud with a 125 MHz reference clock!
- `UARTLCR_H = 0x00000070`: Bits 6:5 are `11` (8 data bits) and bit 4 is `1` (FIFOs enabled).
- `UARTCR = 0x00000301`: Bit 0 is `UARTEN` (1), Bit 8 is `TXE` (Transmit Enable), Bit 9 is `RXE` (Receive Enable).

Command 3: Decode Bitfields (`svd /r UART0 UARTFR`)

Remember the mysterious `0x00000090` we saw when reading `0x40070018`? Let's use the `/r` (raw/register decode) flag to decode it:

```
svd /r UART0 UARTFR
```

```
UART0.UARTFR @ 0x40070018: 0x00000090
[7] TXFE    : 1 (Transmit FIFO empty)
[6] RXFF    : 0 (Receive FIFO full)
[5] TXFF    : 0 (Transmit FIFO full)
[4] RXFE    : 1 (Receive FIFO empty)
[3] BUSY    : 0 (UART busy transmitting)
[2] DCD     : 0 (Data carrier detect)
[1] DSR     : 0 (Data set ready)
[0] CTS     : 0 (Clear to send)
```

In a single command, GDB completely decoded the raw hardware byte into human-readable FIFO and serial transmission states!

Now inspect the pin multiplexer for `GPIO0`:

```
svd /r IO_BANK0 GPIO0_CTRL
```

```
IO_BANK0.GPIO0_CTRL @ 0x40028004: 0x00000002
[4:0] FUNCSEL : 2 (Function 2: UART0 TX)
```

This verifies that `GPIO0` has been successfully multiplexed to function as the hardware `UART0 TX` pin.

Command 4: Live Hardware Manipulation via SWD (`svd /w`)

You can also **write** to registers by name using `svd /w`.

While the processor is halted in GDB, look at your physical Pico 2 hardware board. Let's toggle the LED or write to a register directly:

```
svd /w SIO GPIO_OUT_SET 0x00010000
```

You have full, symbolic, hardware-level control over a running target directly through GDB without having an ELF file or compiling a single line of code.

Part 7: Transitioning to Static Analysis in Ghidra

Dynamic analysis in GDB lets us verify hardware state while stepping through execution. However, to understand the overall architecture, algorithms, and logic of a stripped firmware binary, we must perform **static analysis in Ghidra**.

Step 1: Import the Stripped Binary into Ghidra

1. Launch Ghidra: `ghidraRun`
2. Create a new project named `CTF-01`
3. Drag and drop `0x0001b_ctf/CTF-01.bin` into the Active Project window.
4. In the Import dialog:
 - **Language:** Click `...`, search for `Cortex`, and select **ARM:LE:32:Cortex (default)**.
 - Click **Options...**:
 - Change **Block Name** to `.text`
 - Change **Base Address** to `10000000` (XIP Flash base)
 - Click **OK**, then click **OK** to complete the import.
5. Double-click the file to open the CodeBrowser.
6. When prompted to analyze, click **Yes**, accept default analyzers, and click **Analyze**.

Step 2: The Decompiler Before SVD (The Pointer Maze)

Navigate to `main` at address `0x100001E0` in the Ghidra decompiler.

Before adding SVD support, the decompiler output looks like this:

```
void FUN_100001e0(void)
{
    byte extraout_DL;

    FUN_1000308c();
    FUN_1000311c(DAT_10000278);
    FUN_1000311c(DAT_1000027c);
    FUN_1000311c(DAT_10000280);
    FUN_1000311c(DAT_10000284);
    FUN_10003218(DAT_10000288);
    do {
        FUN_10003218(DAT_10000294,extraout_DL);
        FUN_10003218(DAT_100002a0,extraout_DL);
        FUN_1000311c(DAT_100002a4);
        FUN_10003218(DAT_10000288);
        FUN_10000cc4(1000);
    } while( true );
}
```

Now navigate to subroutine `FUN_100002a8` (`gpio_set_function`):

```
void FUN_100002a8(uint param_1, uint param_2)
{
    *(uint*)(0x40038000 + param_1 * 4 + 4) =
        *(uint*)(0x40038000 + param_1 * 4 + 4) & 0xffffffff7f | 0x40;
    *(uint*)(0x40028000 + param_1 * 8 + 4) = param_2;
    return;
}
```

Look at that decompilation:

- `*(uint*)(0x40038000 + param_1 * 4 + 4)`
- `*(uint*)(0x40028000 + param_1 * 8 + 4) = param_2`

Ghidra doesn't know that `0x40038000` is `PADS_BANK0`, nor that `0x40028000` is `IO_BANK0`. In fact, if you double-click `0x40038000`, Ghidra warns that the address is **unmapped** because the `.bin` import only created a memory block for flash (`0x10000000`).

Part 8: Installing and Running SVD-Loader in Ghidra

Step 1: Install SVD-Loader-Ghidra

`SVD-Loader-Ghidra` is an open-source Ghidra script created by Leveldown Security. It parses CMSIS-SVD files and automatically reconstructs microcontroller memory maps and data structures inside Ghidra.

Open a PowerShell terminal and clone the repository:

```
cd "$env:USERPROFILE"
git clone https://github.com/leveldown-security/SVD-Loader-Ghidra.git
```

[!IMPORTANT] **Ghidra 11/12+ Runtime Compatibility Fix (`#@runtime Jython`):** Modern Ghidra versions default to `PyGhidra` (CPython 3) for `.py` scripts. If Ghidra was not launched via `pyghidraRun`, running `SVD-Loader.py` will fail with: `Unable to load script: SVD-Loader.py - detail: Ghidra was not started with PyGhidra. Python is not available`

`SVD-Loader` was developed for Ghidra's built-in **Jython** interpreter. To instruct Ghidra to use the built-in Jython engine, ensure `#@runtime Jython` is present at the top of `SVD-Loader.py`:

```
# Load specified SVD and generate peripheral memory maps & structures.
#@runtime Jython
#@author Thomas Roth <thomas.roth@leveldown.de>, Ryan Pavlik <ryan.pavlik@gmail.com>
```

(You can add this line using any text editor, or in Ghidra by right-clicking `SVD-Loader.py` in the *Script Manager* and selecting **Edit with basic editor**).

Step 2: Add the Script to Ghidra Script Manager

1. In Ghidra's CodeBrowser, open the **Script Manager**:
 - Go to menu **Window -> Script Manager** (or click the **Script Manager** toolbar icon).

[!NOTE] **Script Manager Toolbar Icons Explained:**

- **New Script (White Paper Icon):** To create a new script from scratch directly in Ghidra, you click the **piece of white paper** ("Create New Script") icon on the toolbar. Ghidra then prompts you to choose the script type: `PyGhidra` (Python 3 in Ghidra 11+), `Java`, or `Jython` (Python 2.7).
- **Manage Script Directories (Folder with List Icon):** Because `SVD-Loader` is an existing multi-file package that relies on the bundled `cmsis_svd` parser library, we do not need to create a blank script. Instead, we register its cloned directory.

2. In the top-right toolbar of the Script Manager window, click the **Manage Script Directories** icon (looks like a small folder with a list).
3. In the "Ghidra Script Directories / Bundle Manager" window that appears, click the **Display file chooser to add bundles to list** icon (the green `+` / folder icon on the top right).
4. Browse to and select your cloned directory: `C:\Users\<username>\SVD-Loader-Ghidra`
5. Click **OK / Select**, then close the Script Directories window.

(Alternatively, you can copy both `SVD-Loader.py` and the `cmsis_svd` folder directly into your default `~/ghidra_scripts` or `C:\Users\<username>\ghidra_scripts` directory, which Ghidra discovers automatically).

Step 3: Run SVD-Loader

1. In the Script Manager search filter box, type: `SVD`
2. Locate `SVD-Loader.py` in the list.
3. Check the checkbox in the **In Tool** column next to `SVD-Loader.py`. This binds `SVD-Loader` directly to your CodeBrowser toolbar and menu for convenient access!
4. Select `SVD-Loader.py` and click the green **Run Script** button in the top right (or double-click the script entry).
5. A file picker dialog opens:
 - Navigate to: `C:\Users\<username>\.svd\rp2350.svd`
 - Click **Open**.

What SVD-Loader Does Automatically in Ghidra

1. Memory Blocks: Creates mapped, volatile memory blocks for `UART0`, `IO_BANK0`, `PADS_BANK0`, `SIO`, `CLOCKS`, and `RESETS`.
2. Symbol Labels: Creates global symbol labels at the exact address of every register (e.g., `UART0_UARTFR`).
3. C Structs: Generates full peripheral data structures in the Data Type Manager (e.g., `struct UART0_Type`).

Check the Ghidra Console window at the bottom of the screen. You will see:

```
Loaded SVD: rp2350.svd
Created peripheral block: SIO at 0xd0000000 (size: 0x1000)
Created peripheral block: PADS_BANK0 at 0x40038000 (size: 0x1000)
Created peripheral block: IO_BANK0 at 0x40028000 (size: 0x1000)
Created peripheral block: UART0 at 0x40070000 (size: 0x1000)
...
Successfully imported all peripherals!
```

Step 4: Re-Run Auto-Analysis to Propagate References

When you initially imported `CTF-01.bin`, Ghidra performed auto-analysis against only the initial Flash block (`0x10000000`). Now that `SVD-Loader.py` has created all 52 on-chip peripheral memory blocks, re-run analysis so Ghidra evaluates references against the newly created regions:

1. Click menu **Analysis -> Auto Analyze 'CTF-01.bin'...** (or press keyboard shortcut `A`).
2. Ensure **Reference**, **Subroutine References**, and **Constant Reference Analyzer** are enabled.
3. Click **Analyze**.

Part 9: Decompiler Transformation: Before and After

Now that the SVD structures are loaded, let's examine subroutine `FUN_100002a8` (`gpio_set_function`) again.

Side-by-Side Decompilation Comparison

```

+-----+
| Decompilation of gpio_set_function()
|
| BEFORE SVD-Loader:
| void FUN_100002a8(uint param_1, uint param_2)
| {
|     *(uint*)(0x40038000 + param_1 * 4 + 4) =
|         *(uint*)(0x40038000 + param_1 * 4 + 4) & 0xffffffff7f
|         | 0x40;
|     *(uint*)(0x40028000 + param_1 * 8 + 4) = param_2;
|     return;
| }
|
| AFTER SVD-Loader:
| void gpio_set_function(uint gpio, uint fn)
| {
|     PADS_BANK0->GPIO[gpio] =
|         (PADS_BANK0->GPIO[gpio] & ~PADS_BANK0_OD)
|         | PADS_BANK0_IE;
|     IO_BANK0->GPIO[gpio].CTRL = fn; /* 2 = UART0 */
|     return;
| }
+-----+

```

Look at the difference:

1. `*(uint*)(0x40038000 + param_1 * 4 + 4)` is recognized as indexing into `PADS_BANK0` electrical pad controls.
2. `& 0xffffffff7f | 0x40` is clearly revealed as clearing the `OD` (Output Disable) bit and setting the `IE` (Input Enable) bit.
3. `*(uint*)(0x40028000 + param_1 * 8 + 4) = param_2` immediately resolves in Ghidra's decompiler to:

```
(&Peripherals::IO_BANK0.GPI00_CTRL)[param_1 * 2] = param_2;
```

Why `[param_1 * 2]`? In RP2350's `IO_BANK0`, each GPIO pin has two 32-bit registers (8 bytes total): `GPI0x_STATUS` (offset +0) and `GPI0x_CTRL` (offset +4). Because `GPI00_CTRL` is a pointer to a 4-byte `uint32_t`, indexing by `[param_1 * 2]` steps forward by 2×4 bytes = 8 bytes per pin, landing directly on each pin's `CTRL` register to assign `param_2` (2 for `UART0`)!

[!NOTE] **Understanding Assembly Listing vs. Decompiler Resolution:** You may notice that in the raw disassembly Listing view, line `100002be` still appears as:

```
100002be 00 f1 80 40    add.w    r0, r0, #0x40000000
```

Why does `#0x40000000` not resolve to a peripheral label here?

- **Arithmetic immediates vs. Memory Operands:** `add.w` is an ALU integer addition, not a load or store instruction. In assembly listings, immediate scalar constants remain literal numbers.
- **Intermediate Math vs. Target Address:** `0x40000000` is the APB/AHB bridge base. The actual peripheral register address (`0x40028004` for `IO_BANK0_GPI00_CTRL`) is calculated dynamically at runtime by adding the pin index offset ($gpio \times 8$), bridge base (`0x40000000`), peripheral offset (`0x28000`), and register offset (`+4`).
- **Where Resolution Appears:** Ghidra resolves this in the **Decompiler window** via data-flow analysis, and in the Listing window as **XREF** annotations on the subsequent `str`/`ldr` instructions that dereference the calculated pointer. If you want `#0x40000000` to show a name in the Listing, right-click the number and select **Set Equate...** (press `E`) to label it `PERIPHERALS_BASE`.

Exploring Structs in the Data Type Manager

In Ghidra's **Data Type Manager** panel (bottom-left):

1. Expand the tree node for `CTF-01.bin`.
2. Expand the `rp2350.svd` category.
3. Locate `UART0_Type`:
 - Double-click `UART0_Type` to open Ghidra's Structure Editor.
 - You can see every field, its byte offset, and its data type:
 - `0x000`: `UARTDR (uint32_t)`
 - `0x018`: `UARTFR (uint32_t)`
 - `0x024`: `UARTIBRD (uint32_t)`
 - `0x028`: `UARTFBRD (uint32_t)`
 - `0x02c`: `UARTLCR_H (uint32_t)`
 - `0x030`: `UARTCR (uint32_t)`

You can apply these struct types to any pointer in Ghidra by right-clicking a variable in the decompiler and selecting **Retype Variable** -> `UART0_Type *`.

Part 10: The Complete Hardware-Aware Reverse Engineering Workflow

By combining SVD in both GDB and Ghidra, you achieve a seamless reverse engineering loop:

The Hardware-Aware Reverse Engineering Loop

1. GHIDRA (Static Analysis + rp2350.svd)

- Identifies functions, call graph, and MMIO
- Pinpoints exact register addresses to watch

|
v

2. GDB + OpenOCD (Dynamic Analysis + rp2350.svd)

- Sets hardware breakpoint (hb *0x100001e0)
- Steps through instructions with si / ni
- Inspects peripheral bitfields live (svd /r)

|
v

3. LIVE HARDWARE INTERACTION

- Validates UART console telemetry (115200 8N1)
- Proves binary patches on physical silicon

Applying the Loop to Solve CTF-01

1. Locate Defects via Ghidra Static Analysis:

- In `main()` (`0x100001E0`), identify Compare Site A at `0x100001FC` (`cmp r3, #94`) and Compare Site B at `0x1000020A` (`cmp r3, #94`).
- Notice that while 0.87 Hz is dangerously high, it passes because $87 \leq 94$.
- To enforce the 0.60 Hz safety limit, $x < 60$ is equivalent to $x \leq 59$. The comparison immediate must be patched from `0x5E` (94) to `0x3B` (59).

2. Locate the Quarantined Flag in Flash:

- Trace the pointer loaded at address `0x100001E8` to address `0x100037A0`.
- Inspecting that memory reveals the token: `"WORLDGRID:BLACKSTART:GRID-7:WATER-3"`.

3. Patch and Export in Ghidra:

- Use the **Bytes Window** workflow to prevent ARM Thumb IT-block context conflicts:
 1. Open the Bytes window (**Window -> Bytes: CTF-01.bin**).
 2. In the Bytes window toolbar, click the **pencil icon (Toggle Edit Mode)**.
 3. In the Listing window, click `0x100001FC` and press **C (Clear Code Bytes)**.
 4. In the Bytes window at offset `100001fc`, click on byte `5E` and change it to `3B`.
 5. In the Listing window, click back on `0x100001FC` and press **D (Disassemble)**.
 6. Repeat at `0x1000020A`: click `0x1000020A` in the Listing, press **C**, change `5E` to `3B` in the Bytes window, click back in the Listing, and press **D**.
- Export the patched binary as `CTF-01_fixed.bin` via **File -> Export Program -> Format: Raw Bytes**.

4. Verify on Live Hardware via OpenOCD & GDB:

- Flash the patched binary to the Pico 2.
- Attach your USB-UART adapter to GPIO 0 (TX) and GPIO 1 (RX) at 115200 baud.
- Observe the corrected, truthful telemetry:

```
GRID STATUS: CRITICAL
DISPATCH PATH: HELD
LAST FRAME: QUARANTINED
RESPONSE>
```

Part 11: Summary, Cheatsheets & Review

GDB Raw Binary Debugging Cheatsheet

Task	Command	Description
Set Architecture	<code>set architecture armv8-m.main</code>	Configures GDB for ARM Cortex-M33 cores.
Connect to OpenOCD	<code>target extended-remote :3333</code>	Connects to OpenOCD debug server.
Reset & Halt	<code>monitor reset halt</code>	Sends reset signal and halts CPU at vector table.

Task	Command	Description
Read Vector Table	<code>x/4wx 0x10000000</code>	Displays Initial Stack Pointer and Reset Vector.
Disassemble at PC	<code>x/10i \$pc</code>	Disassembles 10 instructions at current Program Counter.
Disassemble Range	<code>disassemble 0x100001e0, 0x10000216</code>	Disassembles instructions between two hex addresses.
Hardware Breakpoint	<code>hb *0x100001e0</code>	Sets hardware breakpoint on read-only flash memory.
Read Memory Word	<code>x/wx 0x40070018</code>	Reads one 32-bit hexadecimal word from memory.
Write Memory Word	<code>set *0xd0000014 = 0x10000</code>	Writes 32-bit value directly to memory address.

PyCortexMDebug SVD Cheatsheet

Task	Command	Description
Load Plugin	<code>source ~/PyCortexMDebug/scripts/gdb.py</code>	Imports PyCortexMDebug into GDB Python engine.
Load SVD File	<code>svd_load ~/.svd/rp2350.svd</code>	Parses chip peripheral XML definition.
List Peripherals	<code>svd</code>	Lists all on-chip hardware peripheral blocks.
Dump Peripheral	<code>svd UART0</code>	Reads and displays all registers in a peripheral.
Decode Bitfields	<code>svd /r UART0 UARTFR</code>	Decodes individual bitfields and named flags.
Write Register	<code>svd /w SIO GPIO_OUT_SET 0x10000</code>	Writes to peripheral register by symbolic name.

RP2350 Peripheral Memory Map Quick Reference

Peripheral	Base Address	Size	Primary Purpose
XIP_FLASH	0x10000000	Up to 16MB	External QSPI Flash execution memory
SRAM	0x20000000	512KB	On-chip data memory (stack, heap, .data, .bss)
RESETS	0x40020000	4KB	Subsystem reset controller
IO_BANK0	0x40028000	4KB	GPIO pin function multiplexing (FUNCSEL)
PADS_BANK0	0x40038000	4KB	Electrical drive strength, pulls, and enables
UART0	0x40070000	4KB	Serial communication interface 0 (115200 8N1)
UART1	0x40078000	4KB	Serial communication interface 1
SIO	0xd0000000	4KB	Single-cycle I/O fast GPIO controls

Key Takeaways

1. **A raw `.bin` is not an ELF:** It has no headers, no symbols, and no entry point metadata. You must supply the base address (`0x10000000`) when flashing and debug using explicit addresses.
2. **Flash requires hardware breakpoints:** External flash is read-only during execution. Always use `hb *address` instead of software breakpoints (`b`).
3. **The Vector Table tells all:** Even without symbols, `x/4wx 0x10000000` gives you the initial stack pointer and the reset vector within seconds.
4. **SVD bridges the hardware gap:** CMSIS-SVD turns raw hexadecimal registers into named peripherals, registers, and bitfields across both GDB and Ghidra.
5. **GDB handles live dynamic manipulation:** `PyCortexMDebug` lets you inspect peripheral registers live, decode bitfields with `svd /r`, and toggle pins directly over SWD using `svd /w`.
6. **Ghidra handles static comprehension:** `SVD-Loader-Ghidra` creates mapped memory blocks and C structs, converting raw pointer arithmetic into readable code.
7. **Arithmetic instructions remain arithmetic:** Instructions like `add.w r0, r0, #0x40000000` calculate base addresses across multiple steps; resolution appears in the decompiler and at the load/store instructions that dereference the address.

Glossary

Term	Definition
CMSIS-SVD	Cortex Microcontroller Software Interface Standard - System View Description; an XML format describing microcontroller hardware peripherals.
DWARF	Standardized debugging data format embedded in ELF binaries that maps machine code to source code lines and symbols.
Hardware Breakpoint (hb)	A breakpoint implemented using dedicated CPU comparator registers, required for debugging code in read-only flash memory.
MMIO	Memory-Mapped Input/Output; a hardware architecture where peripheral registers are mapped into the CPU's regular memory address space.
MSP	Main Stack Pointer; the primary ARM Cortex-M stack pointer register.
OpenOCD	Open On-Chip Debugger; a software bridge that connects GDB to physical hardware via a Debug Probe.
PyCortexMDebug	A GDB Python extension that parses SVD files to provide live peripheral inspection and register manipulation.
Reset Vector	The address stored at offset <code>0x00000004</code> in the vector table that points to the first instruction executed upon CPU reset.
SIO	Single-Cycle I/O; a dedicated RP2350 hardware block providing zero-wait-state GPIO manipulation.
SVD-Loader	A Ghidra script that imports SVD files to create memory blocks, symbol labels, and C structs for decompilation.
SWD	Serial Wire Debug; a two-wire physical debug protocol (SWCLK, SWDIO) used to debug ARM microcontrollers.

Term	Definition
UART	Universal Asynchronous Receiver-Transmitter; a physical hardware communication protocol used for serial data transfer.
XIP	eXecute In Place; running code directly from external flash memory without copying it to RAM first.

Happy Hacking!